REVISION CHEAT SHEET:

SQL Server Temporary Data Structures

COMPLETE INTERVIEW-READY NOTES - #TEMP - @TABLE - CTE - TVP - VALUES - TEMPDB

TOPIC & SUBTOPIC INDEX

Compact study order: What Is a Temporary Data Structure? -> Main Temporary Structures -> Temporary Tables -> Local vs Global Temp Tables -> Indexes, Constraints & Statistics -> Scope & Stored Procedures -> Table Variables. Related details continue in the same flow without unnecessary blank pages.

- What Is a Temporary Data Structure?
- Main Temporary Structures
- Temporary Tables
- Local vs Global Temp Tables
- Indexes, Constraints & Statistics
- Scope & Stored Procedures
- Table Variables
- Temp Table vs Table Variable
- Common Table Expressions
- Recursive CTE & MAXRECURSION
- Derived Tables
- Table-Valued Parameters
- VALUES Table Constructor
- Choosing the Correct Structure
- tempdb, Statistics & Indexing
- Concurrency & Performance
- Practical FundraisingDb Examples
- Common Beginner Mistakes
- Best Practices
- Final Memory Diagram

Temporary Data Structures in SQL Server

Topic Structure

EXAMPLE / FLOW

```
Temporary Data Structures
        →
Temporary Tables
        →
Local Temporary Table
        →
Global Temporary Table
        →
Table Variables
        →
CTEs
        →
Derived Tables
        →
Table-Valued Parameters
        →
VALUES Table Constructor
        →
Choosing the Correct Structure
        →
Performance / tempdb / Statistics / Indexing
```

What Is a Temporary Data Structure?

A **temporary data structure** holds data only while SQL Server is performing some temporary work. We use it when intermediate data is needed for calculations, filtering, reporting, stored procedures, or multi-step processing.

The important point is that temporary does **not always mean memory-only**. Temporary tables and table variables can use the SQL Server `tempdb` system database.

Simple Flow

EXAMPLE / FLOW

```
Permanent Data → Temporary Processing → Final Result
```

Example:

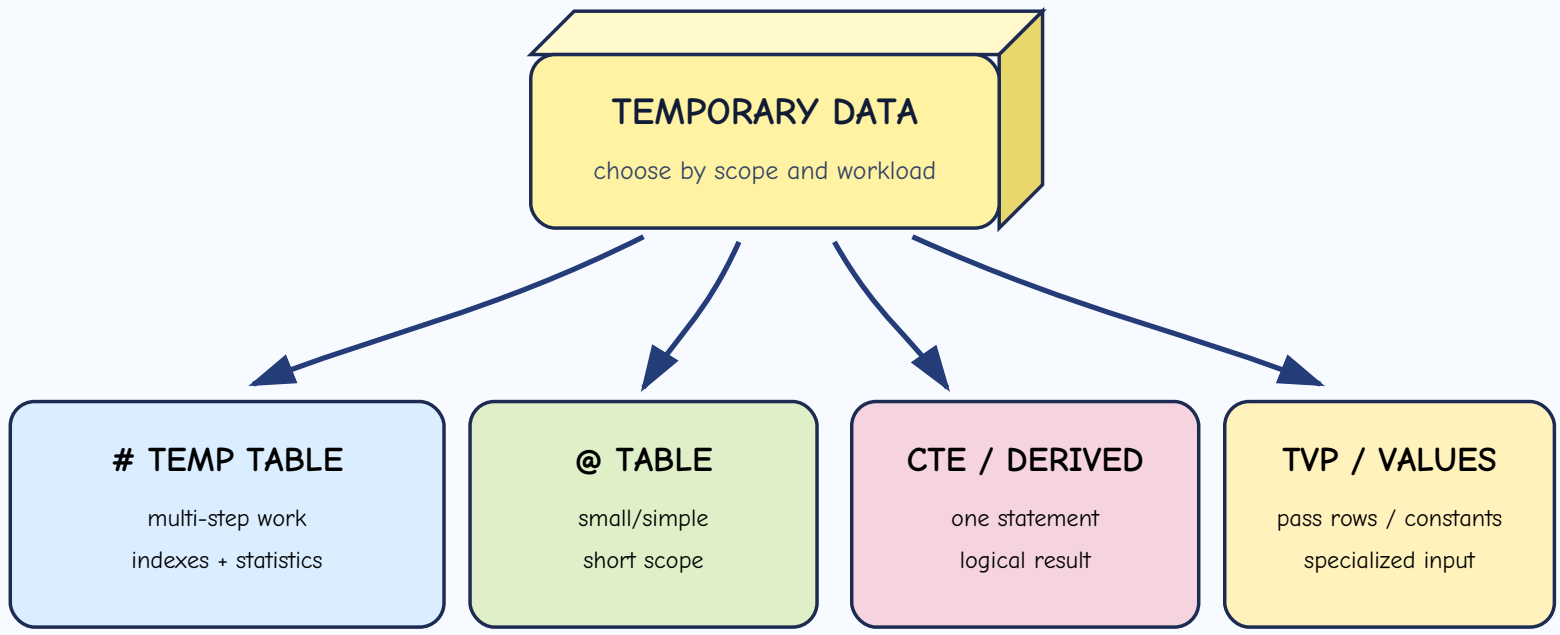
EXAMPLE / FLOW

```
fundraising.Gift
        →
#LargeGifts
        →
Calculate / Filter / Join
        →
Return Report
```

Main Temporary Structures

3D DIAGRAM

Temporary Data Structure Map



Temporary does not mean one single object. Pick the structure that matches scope, reuse, row volume, and optimizer needs.

Structure	Syntax	Main Purpose
Local temporary table	`#TableName`	Temporary multi-step processing
Global temporary table	`##TableName`	Share temporary data between sessions
Table variable	`@TableName`	Small temporary working data
CTE	`WITH CTE AS (...)`	Simplify one SQL statement
Derived table	`(SELECT...) AS X`	Inline temporary query
Table-valued parameter	User-defined table type	Pass many rows to procedure
VALUES constructor	`VALUES (...)`	Small inline row set

Temporary Table

1 What Is a Temporary Table?

A **temporary table** works much like a normal SQL Server table, but it is created for temporary processing. SQL Server stores temporary tables in the `tempdb` database.

Temporary tables support columns, indexes, constraints, statistics, `INSERT`, `UPDATE`, `DELETE`, joins, and most normal table operations.

Flow

EXAMPLE / FLOW

```
Create #Table → Insert Data → Process Data → Read Result → Drop Table
```

Types of Temporary Tables

SQL Server mainly provides two types:

EXAMPLE / FLOW

```
#LocalTemporaryTable
        OR
##GlobalTemporaryTable
```

Local Temporary Table

Definition

A **local temporary table** starts with one #.

It belongs to the current SQL Server session. Another independent SQL Server session normally cannot access it.

Microsoft describes local temporary tables as visible only to the current session.

Syntax

SQL

```sql
CREATE TABLE #GiftWork
(
    GiftId BIGINT,
    ConstituentId BIGINT,
    TotalAmount DECIMAL(19,4)
);
```

Local Temporary Table Example

Suppose we want to temporarily store large donations.

SQL

```sql
CREATE TABLE #LargeGifts
(
    GiftId BIGINT,
    ConstituentId BIGINT,
    GiftDate DATE,
    TotalAmount DECIMAL(19,4)
);
```

Insert data from `fundraising.Gift`:

SQL

```sql
INSERT INTO #LargeGifts
(
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
)
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

Read the temporary data:

```sql
SELECT *
FROM #LargeGifts;
```

Why Is It Called Local?

Assume two developers connect to SQL Server.

```
Session 1 → #LargeGifts → Available
Session 2 → #LargeGifts → Different / Not Available
```

SQL Server internally adds a unique suffix to local temporary table names so different sessions can create the same `#TableName` without normally conflicting.

Global Temporary Table

Definition

A **global temporary table** starts with two # characters:

```
##
```

Unlike a local temporary table, it can be visible to other SQL Server sessions.

Syntax

```sql
CREATE TABLE ##DailyGiftSummary
(
    CampaignId BIGINT,
    TotalGiftAmount DECIMAL(19,4)
);
```

Microsoft documents ## tables as global temporary tables that are visible across sessions.

Global Temporary Table Example

```sql
CREATE TABLE ##CampaignTotals
(
    CampaignId BIGINT,
    TotalAmount DECIMAL(19,4)
);

INSERT INTO ##CampaignTotals
(
    CampaignId,
    TotalAmount
)
SELECT
    CampaignId,
    SUM(TotalAmount)
FROM fundraising.Gift
WHERE CampaignId IS NOT NULL
GROUP BY CampaignId;
```

Another session can then run:

```sql
SELECT *
FROM ##CampaignTotals;
```

Local vs Global Temporary Table

Feature	Local #Table	Global ##Table
Prefix	#	##
Current session	Yes	Yes
Other sessions	No	Yes
Stored in tempdb	Yes	Yes
Common application use	Very common	Rare
Concurrency risk	Lower	Higher
Recommended default	Yes	Usually no

Use a local temporary table for most application processing.

Use a global temporary table only when temporary information genuinely must be shared between different sessions.

Creating a Temporary Table

Use CREATE TABLE.

SQL

```
CREATE TABLE #GiftSummary
(
    ConstituentId BIGINT NOT NULL,
    GiftCount INT NOT NULL,
    TotalAmount DECIMAL(19,4) NOT NULL
);
```

This structure exists temporarily but behaves much like a regular table.

Creating a Temporary Table with SELECT INTO

SQL Server can create and populate a temporary table at the same time.

SQL

```
SELECT
    ConstituentId,
    COUNT(*) AS GiftCount,
    SUM(TotalAmount) AS TotalAmount
INTO #GiftSummary
FROM fundraising.Gift
GROUP BY ConstituentId;
```

Here SQL Server determines the column definitions from the SELECT.

Flow

EXAMPLE / FLOW

```
SELECT Data → INTO #TempTable → Table Created + Data Inserted
```

INSERT Data into Temporary Table

After creating the table, use normal INSERT.

SQL

```
INSERT INTO #GiftSummary
(
    ConstituentId,
    GiftCount,
    TotalAmount
)
SELECT
    ConstituentId,
    COUNT(*),
```

```
    SUM(TotalAmount)
FROM fundraising.Gift
GROUP BY ConstituentId;
```

The syntax is the same idea as inserting into a permanent table.

UPDATE Temporary Table

Temporary tables support normal updates.

SQL

```
UPDATE #GiftSummary
SET TotalAmount = TotalAmount + 100
WHERE ConstituentId = 101;
```

Only the data in the temporary table changes. The original `fundraising.Gift` rows are not changed.

DELETE from Temporary Table

SQL

```
DELETE FROM #GiftSummary
WHERE TotalAmount < 100;
```

The rows are removed only from the temporary table.

SELECT from Temporary Table

SQL

```
SELECT
    ConstituentId,
    GiftCount,
    TotalAmount
FROM #GiftSummary;
```

You can filter, sort, aggregate, and join temporary tables exactly as needed.

Joining Temporary Tables with Permanent Tables

Temporary tables become especially useful when their results must be joined to your real business tables.

SQL

```
SELECT
    c.DisplayName,
    gs.GiftCount,
    gs.TotalAmount
FROM #GiftSummary AS gs
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = gs.ConstituentId;
```

Flow

EXAMPLE / FLOW

```
fundraising.Gift → #GiftSummary → JOIN Constituent → Final Report
```

Indexes on Temporary Tables

Temporary tables support indexes.

Indexes are useful when the temporary table contains many rows and SQL Server frequently searches, joins, or sorts those rows.

```sql
CREATE INDEX IX_GiftSummary_ConstituentId
ON #GiftSummary(ConstituentId);
```

Clustered Index on Temporary Table

```sql
CREATE CLUSTERED INDEX CIX_LargeGifts_GiftDate
ON #LargeGifts(GiftDate);
```

A clustered index controls the main physical index structure of the table.

Use one when its ordering/access pattern genuinely helps your temporary workload.

Nonclustered Index on Temporary Table

```sql
CREATE NONCLUSTERED INDEX IX_LargeGifts_Constituent
ON #LargeGifts(ConstituentId);
```

This can help queries such as:

```sql
SELECT *
FROM #LargeGifts
WHERE ConstituentId = 1001;
```

When Should You Index a Temporary Table?

Do not automatically index every temporary table.

Indexes help reads but require extra work during `INSERT`, `UPDATE`, and `DELETE`.

Simple Rule

```
Few rows + used once → Usually no extra index

Many rows + repeated searches/joins → Consider index
```

Constraints on Temporary Tables

Temporary tables can use normal data integrity rules such as:

```
PRIMARY KEY
UNIQUE
CHECK
DEFAULT
NOT NULL
```

Example:

```sql
CREATE TABLE #GiftWork
(
    GiftId BIGINT NOT NULL PRIMARY KEY,
    TotalAmount DECIMAL(19,4)
        CHECK (TotalAmount > 0),
    Processed BIT NOT NULL DEFAULT 0
);
```

Temporary tables support constraint definitions, but `FOREIGN KEY` constraints on temporary tables aren't enforced in the normal way; SQL Server skips such foreign-key definitions with a warning.

PRIMARY KEY on Temporary Table

```SQL
CREATE TABLE #GiftIds
(
    GiftId BIGINT PRIMARY KEY
);
```

A primary key prevents duplicate `GiftId` values and also gives SQL Server an index that can help searching.

UNIQUE Constraint

```SQL
CREATE TABLE #CampaignWork
(
    CampaignId BIGINT PRIMARY KEY,
    CampaignCode VARCHAR(50) UNIQUE
);
```

`UNIQUE` prevents duplicate values in the specified key.

CHECK Constraint

```SQL
CREATE TABLE #AllocationWork
(
    GiftId BIGINT,
    FundId BIGINT,
    AllocationAmount DECIMAL(19,4)
        CHECK (AllocationAmount > 0)
);
```

This matches the business idea in your `GiftAllocation` design that allocation amounts must be positive.

Temporary Table Constraint Naming

Inside highly concurrent stored procedures, avoid unnecessarily giving temporary-table constraints fixed explicit names.

SQL Server isolates local temporary table names, but explicitly named constraints can create name conflicts in certain concurrent situations. Microsoft specifically documents this concurrency issue.

Prefer:

```SQL
CREATE TABLE #GiftWork
(
    GiftId BIGINT PRIMARY KEY
);
```

instead of unnecessarily doing:

```SQL
CONSTRAINT PK_GiftWork PRIMARY KEY
```

inside heavily concurrent temporary workflows.

Statistics on Temporary Tables

One major advantage of temporary tables is that SQL Server can use statistics for query optimization.

Statistics help SQL Server estimate:

EXAMPLE / FLOW

```
How many rows?
        →
Which join?
        →
Which index?
        →
How much memory?
        →
Which execution plan?
```

Temporary tables provide normal statistics capabilities that table variables do not.

Example of Why Statistics Matter

Suppose:

EXAMPLE / FLOW

```
#GiftWork contains 5 rows
```

versus:

EXAMPLE / FLOW

```
#GiftWork contains 5,000,000 rows
```

Those two cases may need completely different execution plans.

Statistics help SQL Server understand the data size and distribution more accurately.

Dropping Temporary Tables

Use:

SQL

```
DROP TABLE #GiftWork;
```

Modern SQL Server also supports:

SQL

```
DROP TABLE IF EXISTS #GiftWork;
```

This is useful because no error occurs when the table does not exist.

Temporary Table Scope

Local Temporary Table Scope

A local temporary table normally belongs to the session that created it.

EXAMPLE / FLOW

```
Session Starts → #Temp Created → Statements Use It → Session Ends → Removed
```

If a local temporary table is created inside a stored procedure, its scope follows special procedure rules discussed below.

Temporary Table Lifetime

For a normal local temporary table:

EXAMPLE / FLOW

```
Connection Opens → CREATE #Table → Use Table → Connection Closes → Table Removed
```

You may drop it earlier manually:

```sql
DROP TABLE #GiftWork;
```

Global Temporary Table Lifetime

With SQL Server's default global-temp-table auto-drop behavior, a global temporary table remains available while required by the creating session/active references and is eventually removed after those references finish.

Because other sessions can access the same object, global temporary tables require more care than local temporary tables.

Temporary Table Inside a Stored Procedure

Temporary tables are extremely common inside stored procedures.

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetLargeGiftReport
AS
BEGIN
    SET NOCOUNT ON;

    CREATE TABLE #LargeGifts
    (
        GiftId BIGINT,
        ConstituentId BIGINT,
        TotalAmount DECIMAL(19,4)
    );

    INSERT INTO #LargeGifts
    SELECT
        GiftId,
        ConstituentId,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000;

    SELECT *
    FROM #LargeGifts;
END;
```

Stored Procedure Scope Rule

If a temporary table is created **inside** a stored procedure:

```
Procedure Starts → CREATE #Temp → Procedure Uses It → Procedure Ends → #Temp Removed
```

A nested stored procedure called from that procedure can access the parent's local temporary table, but the caller cannot continue using a temp table that was created only inside the completed procedure.

Temporary Table Created Before Procedure Call

Example:

```sql
CREATE TABLE #SelectedGifts
(
    GiftId BIGINT
);

INSERT INTO #SelectedGifts
VALUES (1), (2), (3);

EXEC fundraising.usp_ProcessSelectedGifts;
```

A procedure executing in the same connection can potentially use that existing local temporary table.

This is sometimes useful but creates tight coupling between procedure implementations, so use the pattern carefully.

Temporary Tables Inside Transactions

Temporary-table changes can participate in SQL Server transactions.

Example:

SQL

```
CREATE TABLE #GiftWork
(
    GiftId BIGINT,
    TotalAmount DECIMAL(19,4)
);

INSERT INTO #GiftWork
VALUES (1, 1000);

BEGIN TRANSACTION;

UPDATE #GiftWork
SET TotalAmount = 2000
WHERE GiftId = 1;

ROLLBACK TRANSACTION;

SELECT *
FROM #GiftWork;
```

After the rollback, the temporary table value returns to the state before the rolled-back update.

Transaction Flow

EXAMPLE / FLOW

```
#GiftWork = 1000
        →
BEGIN TRANSACTION
        →
UPDATE = 2000
        →
ROLLBACK
        →
#GiftWork = 1000
```

Transactions begin with `BEGIN TRANSACTION` and finish using `COMMIT` or `ROLLBACK`.

When to Use Temporary Tables

Use a temporary table when:

- intermediate results are reused several times;
- many rows are involved;
- indexes can improve performance;
- statistics are important;
- several SQL statements must work with the same intermediate data;
- a complicated query becomes easier when broken into steps.

Example:

EXAMPLE / FLOW

```
Gift → Filter → #GiftWork → Aggregate → Join → Final Report
```

When Not to Use Temporary Tables

Do not automatically create temporary tables for every query.

A simple query may work better using a direct query, CTE, or derived table.

Avoid:

EXAMPLE / FLOW

```
Simple SELECT → unnecessary #Table → SELECT again
```

when this works:

EXAMPLE / FLOW

```
Simple SELECT → Final Result
```

Table Variables

What Is a Table Variable?

A **table variable** is a variable whose value has a table structure.

Its name starts with:

EXAMPLE / FLOW

```
@
```

Example:

EXAMPLE / FLOW

```
@GiftWork
```

Table variables can also use `tempdb`; they should not be thought of as objects guaranteed to remain only in RAM.

Declaring a Table Variable

A table variable uses `DECLARE`.

SQL

```sql
DECLARE @GiftWork TABLE
(
    GiftId BIGINT,
    ConstituentId BIGINT,
    TotalAmount DECIMAL(19,4)
);
```

There is no separate `CREATE TABLE @GiftWork`.

Declaration Flow

EXAMPLE / FLOW

```
DECLARE @Table → Insert Data → Process Data → Scope Ends → Gone
```

Inserting into a Table Variable

SQL

```sql
DECLARE @LargeGifts TABLE
(
    GiftId BIGINT,
    ConstituentId BIGINT,
```

```
    TotalAmount DECIMAL(19,4)
);

INSERT INTO @LargeGifts
(
    GiftId,
    ConstituentId,
    TotalAmount
)
SELECT
    GiftId,
    ConstituentId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

Selecting from a Table Variable

SQL

```
SELECT *
FROM @LargeGifts;
```

It looks similar to querying a normal table.

Updating a Table Variable

SQL

```
UPDATE @LargeGifts
SET TotalAmount = TotalAmount + 50
WHERE GiftId = 100;
```

Normal **UPDATE** syntax works.

Deleting from a Table Variable

SQL

```
DELETE FROM @LargeGifts
WHERE TotalAmount < 100;
```

Again, the syntax is similar to a normal table.

Table Variable Scope

A table variable has variable-like scope.

For example:

SQL

```
DECLARE @GiftWork TABLE
(
    GiftId BIGINT
);

SELECT *
FROM @GiftWork;
```

Once the current batch/procedure scope ends, `@GiftWork` is no longer available.

Table Variable Lifetime

EXAMPLE / FLOW

```
DECLARE @GiftWork
        →
Use inside current scope
        →
Scope finishes
        →
@GiftWork disappears
```

You normally do not execute:

SQL

```
DROP TABLE @GiftWork;
```

The variable ends automatically with its scope.

Constraints on Table Variables

Table variables can define important rules during declaration.

A common example is a primary key:

SQL

```
DECLARE @GiftWork TABLE
(
    GiftId BIGINT PRIMARY KEY,
    TotalAmount DECIMAL(19,4) NOT NULL
);
```

Primary key and unique definitions can also provide index structures.

UNIQUE on a Table Variable

SQL

```
DECLARE @CampaignWork TABLE
(
    CampaignId BIGINT PRIMARY KEY,
    CampaignCode VARCHAR(50) UNIQUE
);
```

This prevents duplicate values in `CampaignCode`.

Indexing Table Variables

Table variables do **not** provide the same flexible index management as temporary tables.

You usually define required key/index information when declaring the variable rather than freely adding and removing indexes afterward.

Example:

SQL

```
DECLARE @GiftWork TABLE
(
    GiftId BIGINT
        PRIMARY KEY,
    ConstituentId BIGINT,
    TotalAmount DECIMAL(19,4)
);
```

Modern SQL Server also supports certain inline index definitions, but temporary tables remain more flexible for serious indexing requirements.

Statistics on Table Variables

This is one of the most important differences.

Traditional table variables **do not maintain normal column statistics**.

Modern SQL Server can use **table variable deferred compilation** at database compatibility level 150 or higher. This allows the first compilation to use the actual row count available at that point, but it still does not create normal column statistics.

Table Variable Deferred Compilation

Old simplified idea:

EXAMPLE / FLOW

```
@Table → Poor row estimate → Possibly poor execution plan
```

Modern deferred compilation:

EXAMPLE / FLOW

```
Populate @Table
        →
Compile statement later
        →
See actual initial row count
        →
Better possible plan
```

But:

EXAMPLE / FLOW

```
Actual Row Count ≠ Full Column Statistics
```

So table variables are still not identical to temporary tables.

Limitations of Table Variables

Compared with temporary tables:

- indexing options are more limited;
- normal column statistics are unavailable;
- optimizer information is more limited;
- changing the table definition after declaration is much more restricted;
- they are usually less suitable for large, complicated intermediate workloads.

Use them because the workload fits-not simply because they look shorter.

When to Use Table Variables

A table variable is useful when temporary data is relatively small and processing is simple.

Example:

EXAMPLE / FLOW

```
Small list of GiftIds
        →
Process within one procedure
        →
Finish
```

Example:

SQL

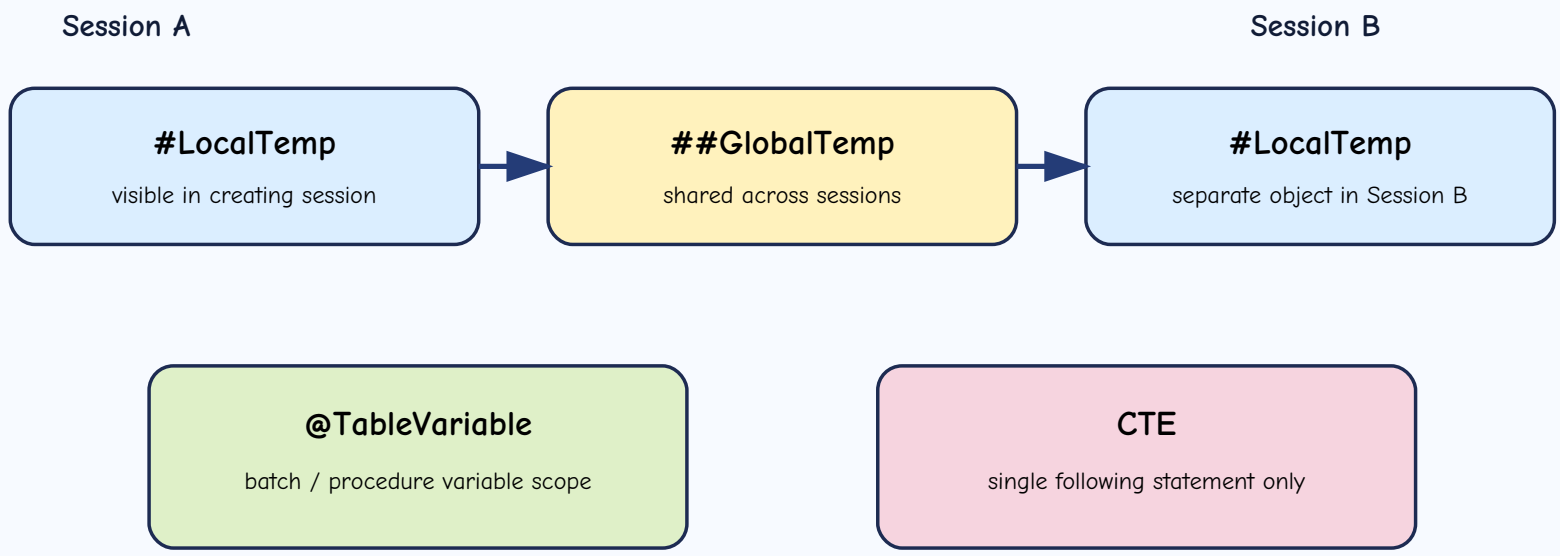
```
DECLARE @GiftIds TABLE
(
    GiftId BIGINT PRIMARY KEY
);

INSERT INTO @GiftIds
VALUES (101), (102), (103);
```

Temporary Table vs Table Variable

3D DIAGRAM

Scope and Lifetime - Who Can See the Data?



Local temp tables isolate sessions; global temp tables are shared; table variables and CTEs have much narrower logical scope.

Feature	#TempTable	@TableVariable
Creation	CREATE TABLE	DECLARE
Stored using tempdb resources	Yes	Can be
Scope	Session/procedure	Batch/procedure variable scope
Normal statistics	Yes	No
Flexible indexes	Yes	Limited
Large data	Usually better	Usually less suitable
Multiple complex operations	Good	Possible but less flexible
Simple small work	Good	Often convenient
Manual drop	Possible	No normal drop required

Memory Rule

EXAMPLE / FLOW

```
Large / complex / need statistics → #TempTable

Small / simple / short-lived → @TableVariable
```

This is a guideline, not an exact row-count law.

Common Table Expression - CTE

What Is a CTE?

A **Common Table Expression** is a named query expression created using `WITH`.

It makes a complex SQL statement easier to read.

A CTE is not a normal temporary table and normally does not store a reusable physical temporary result. Microsoft documents CTE results as not materialized; repeated references can require the underlying expression to be executed again.

Simple CTE Syntax

```sql
;WITH LargeGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
)
SELECT *
FROM LargeGifts;
```

CTE Flow

EXAMPLE / FLOW

```
WITH CTE → Define Query → Main Statement Uses CTE → Statement Ends → CTE Gone
```

Why Is ;WITH Often Used?

You often see:

```sql
;WITH LargeGifts AS
```

The semicolon ends the previous SQL statement.

This prevents syntax problems when the previous statement was not explicitly terminated.

Simple CTE Example with FundraisingDb

```sql
;WITH ConstituentGiftTotals AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
)
SELECT
    c.DisplayName,
    g.TotalGiving
FROM ConstituentGiftTotals AS g
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId;
```

The CTE first calculates donor totals and the outer query adds donor names.

CTE Scope

A CTE normally belongs only to the **single statement immediately following its definition**.

Valid:

```sql
;WITH GiftTotals AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalAmount
    FROM fundraising.Gift
    GROUP BY ConstituentId
```

```
)
SELECT *
FROM GiftTotals;
```

After this statement finishes, `GiftTotals` is not available to another independent statement.

Incorrect CTE Scope Example

SQL

```
;WITH GiftTotals AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalAmount
    FROM fundraising.Gift
    GROUP BY ConstituentId
)
SELECT *
FROM GiftTotals;

-- GiftTotals no longer exists here.
SELECT *
FROM GiftTotals;
```

The second query fails because the CTE scope has ended.

CTE Lifetime

A CTE exists logically only while its associated statement is running.

EXAMPLE / FLOW

```
Define CTE → Execute One Statement → CTE Scope Ends
```

It is much shorter-lived than a temporary table.

Recursive CTE

Definition

A **recursive CTE** references itself.

It is useful for hierarchical data, trees, organizational structures, parent-child relationships, or generating a sequence.

Microsoft describes a recursive CTE as containing an **anchor member** and a **recursive member**.

Recursive CTE Structure

SQL

```
;WITH CTEName AS
(
    -- Anchor query

    UNION ALL

    -- Recursive query referencing CTEName
)
SELECT *
FROM CTEName;
```

Flow

EXAMPLE / FLOW

```
Anchor → Recursive Step → Recursive Step → ... → No More Rows → Stop
```

Recursive CTE Example

Your current FundraisingDb does not contain a parent-child campaign column, so it would be incorrect to invent a hierarchy inside the existing design.

A simple number example clearly demonstrates recursion:

SQL

```
;WITH Numbers AS
(
    SELECT 1 AS NumberValue

    UNION ALL

    SELECT NumberValue + 1
    FROM Numbers
    WHERE NumberValue < 5
)
SELECT *
FROM Numbers;
```

Result:

EXAMPLE / FLOW

```
1
2
3
4
5
```

Recursive CTE Processing

EXAMPLE / FLOW

```
Anchor gives 1
   →
Recursive gives 2
   →
Recursive gives 3
   →
Recursive gives 4
   →
Recursive gives 5
   →
Condition becomes false
   →
Stop
```

MAXRECURSION

A badly written recursive CTE could continue too long.

SQL Server provides:

SQL

```
OPTION (MAXRECURSION 100);
```

Example:

SQL

```
;WITH Numbers AS
(
    SELECT 1 AS NumberValue
```

```
    UNION ALL

    SELECT NumberValue + 1
    FROM Numbers
    WHERE NumberValue < 500
)
SELECT *
FROM Numbers
OPTION (MAXRECURSION 500);
```

Microsoft documents `MAXRECURSION` as a protection/control for recursive queries.

Simple CTE vs Recursive CTE

Simple CTE	Recursive CTE
Does not reference itself	References itself
Simplifies query	Processes hierarchy/recursion
Usually one logical pass	Multiple recursive passes
Common for reporting	Common for tree structures

Temporary Table vs CTE

Feature	#TempTable	CTE
Physical temporary object	Yes	No normal materialized object
Multiple statements	Yes	No
Indexes	Yes	No direct CTE index
Statistics	Temp table statistics	Uses underlying data/statistics
Can update intermediate rows	Yes	Not like a reusable table
Best use	Multi-step processing	One-statement readability

Memory Rule

EXAMPLE / FLOW

```
Need data several times → #TempTable

Need readable logic one time → CTE
```

Derived Table

What Is a Derived Table?

A **derived table** is a query placed inside the **FROM** clause.

It acts like a temporary named result only inside the current query.

Syntax

SQL

```
SELECT *
FROM
(
    SELECT ...
) AS AliasName;
```

Derived Table Example

```sql
SELECT
    c.DisplayName,
    x.TotalGiving
FROM
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
) AS x
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = x.ConstituentId;
```

x exists only inside this query.

Derived Table Flow

EXAMPLE / FLOW

```
Inner SELECT → Derived Result x → Outer SELECT → Result
```

CTE vs Derived Table

These two queries can represent similar logic.

Derived Table

```sql
SELECT *
FROM
(
    SELECT *
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
) AS LargeGifts;
```

CTE

```sql
;WITH LargeGifts AS
(
    SELECT *
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
)
SELECT *
FROM LargeGifts;
```

CTE vs Derived Table Comparison

CTE	Derived Table
Named before main statement	Written directly in FROM
Easier for long logic	Good for short logic
Supports recursion	No self-recursive derived table
Can improve readability	Good for simple inline processing
Statement scope	Query scope

Use whichever makes the SQL clearer.

Table Variable vs CTE

@TableVariable	CTE
Holds temporary rows	Logical query expression
Can insert/update/delete	No reusable stored rows
Can be referenced in several statements within scope	Usually one statement
Can define keys	No direct indexes
Useful for temporary working data	Useful for readable query logic

---

Table-Valued Parameter - TVP

Definition

A **Table-Valued Parameter** allows an application or caller to send multiple rows to a stored procedure as one parameter.

This is especially useful with ASP.NET Core Web APIs when the API must send a collection of rows to SQL Server.

TVPs are based on user-defined table types and must be declared `READONLY` in stored-procedure parameters.

TVP Flow

EXAMPLE / FLOW

```
.NET Collection → Table-Valued Parameter → Stored Procedure → SQL Processing
```

Example business case:

EXAMPLE / FLOW

```
Gift
        →
Several Fund Allocations
        →
Send all allocations together
        →
SQL Server
```

Create a Table Type

SQL

```sql
CREATE TYPE fundraising.GiftAllocationInputType AS TABLE
(
    FundId BIGINT NOT NULL,
    AllocationAmount DECIMAL(19,4) NOT NULL
);
```

This creates a reusable **type definition**, not temporary data itself.

Use TVP in a Stored Procedure

SQL

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_ProcessGiftAllocations
    @GiftId BIGINT,
    @Allocations fundraising.GiftAllocationInputType READONLY
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @GiftId AS GiftId,
        FundId,
        AllocationAmount
```

```
    FROM @Allocations;
END;
```

The **READONLY** keyword is required.

Calling a TVP from T-SQL

SQL

```
DECLARE @Allocations fundraising.GiftAllocationInputType;

INSERT INTO @Allocations
(
    FundId,
    AllocationAmount
)
VALUES
    (1, 700.00),
    (2, 300.00);

EXEC fundraising.usp_ProcessGiftAllocations
    @GiftId = 1001,
    @Allocations = @Allocations;
```

This closely matches your FundraisingDb rule where one gift can be divided among multiple funds.

TVP Limitations

TVPs are input-oriented and **READONLY**.

Inside the receiving procedure, you cannot directly **INSERT**, **UPDATE**, or **DELETE** the incoming TVP itself.

SQL Server also does not maintain normal statistics on TVP columns.

If extensive processing is needed:

EXAMPLE / FLOW

```
TVP → Copy to #TemporaryTable → Index / Modify → Process
```

VALUES Table Constructor

Definition

VALUES lets you create a small row set directly in SQL.

Example:

SQL

```
SELECT *
FROM
(
    VALUES
        (1, 'Cash'),
        (2, 'Check'),
        (3, 'Electronic'),
        (4, 'Other')
) AS GiftTypes(GiftTypeId, GiftTypeName);
```

This matches the simple gift-type values in your FundraisingDb design.

VALUES Flow

EXAMPLE / FLOW

```
VALUES Rows → Temporary Inline Row Set → Query / Join
```

VALUES with INSERT

```sql
INSERT INTO #GiftIds
(
    GiftId
)
VALUES
    (101),
    (102),
    (103);
```

When used directly as **INSERT ... VALUES**, SQL Server supports up to 1,000 rows in one table-value constructor. Used as a derived table, the same 1,000-row limit does not apply in that way.

VALUES in a JOIN

```sql
SELECT
    g.GiftId,
    g.GiftType,
    gt.GiftTypeName
FROM fundraising.Gift AS g
INNER JOIN
(
    VALUES
        (1, 'Cash'),
        (2, 'Check'),
        (3, 'Electronic'),
        (4, 'Other')
) AS gt(GiftTypeId, GiftTypeName)
    ON gt.GiftTypeId = g.GiftType;
```

This can be useful for very small static mappings.

Temporary Result Set

A **temporary result set** is a general concept rather than one special SQL Server object.

A query can produce intermediate rows while SQL Server is executing joins, sorting, aggregation, CTEs, derived tables, or other operations.

SQL Server may internally use work tables or work files in **tempdb** for operations such as sorts, hashes, spools, and intermediate query processing.

In-Memory Temporary Structures

Do not assume:

```
@TableVariable = Always RAM
```

or:

```
#TempTable = Always Disk
```

SQL Server manages memory and storage internally, and both can involve **tempdb**.

For specialized workloads SQL Server also has **memory-optimized table types/tables**, but those belong to In-Memory OLTP and require different configuration. They should be learned after normal temporary tables and table variables.

Local Temp Table vs Global Temp Table

Question	Local	Global
Syntax	`#GiftWork`	`##GiftWork`
Who sees it?	Current session	Other sessions too
Normal application choice	Yes	Rare
Risk of shared-state problems	Low	Higher
Stored in tempdb	Yes	Yes

Prefer local temporary tables unless cross-session sharing is genuinely necessary.

Temp Table vs Table Variable

EXAMPLE / FLOW

```
Need strong optimizer information?
        →
#TempTable

Need several indexes?
        →
#TempTable

Very small/simple local working set?
        →
@TableVariable

Need repeated use across statements?
        →
Either, but #TempTable usually provides more flexibility
```

Temp Table vs CTE

EXAMPLE / FLOW

```
One statement + readability → CTE

Multiple statements → #TempTable

Need indexes → #TempTable

Need intermediate UPDATE → #TempTable

Need recursion → Recursive CTE
```

CTE vs Derived Table

EXAMPLE / FLOW

```
Short inline query → Derived Table

Longer readable query → CTE

Recursive processing → CTE
```

Performance is not automatically better just because one syntax is used. The final execution plan matters.

Table Variable vs CTE

EXAMPLE / FLOW

```
Need to store temporary rows → @TableVariable

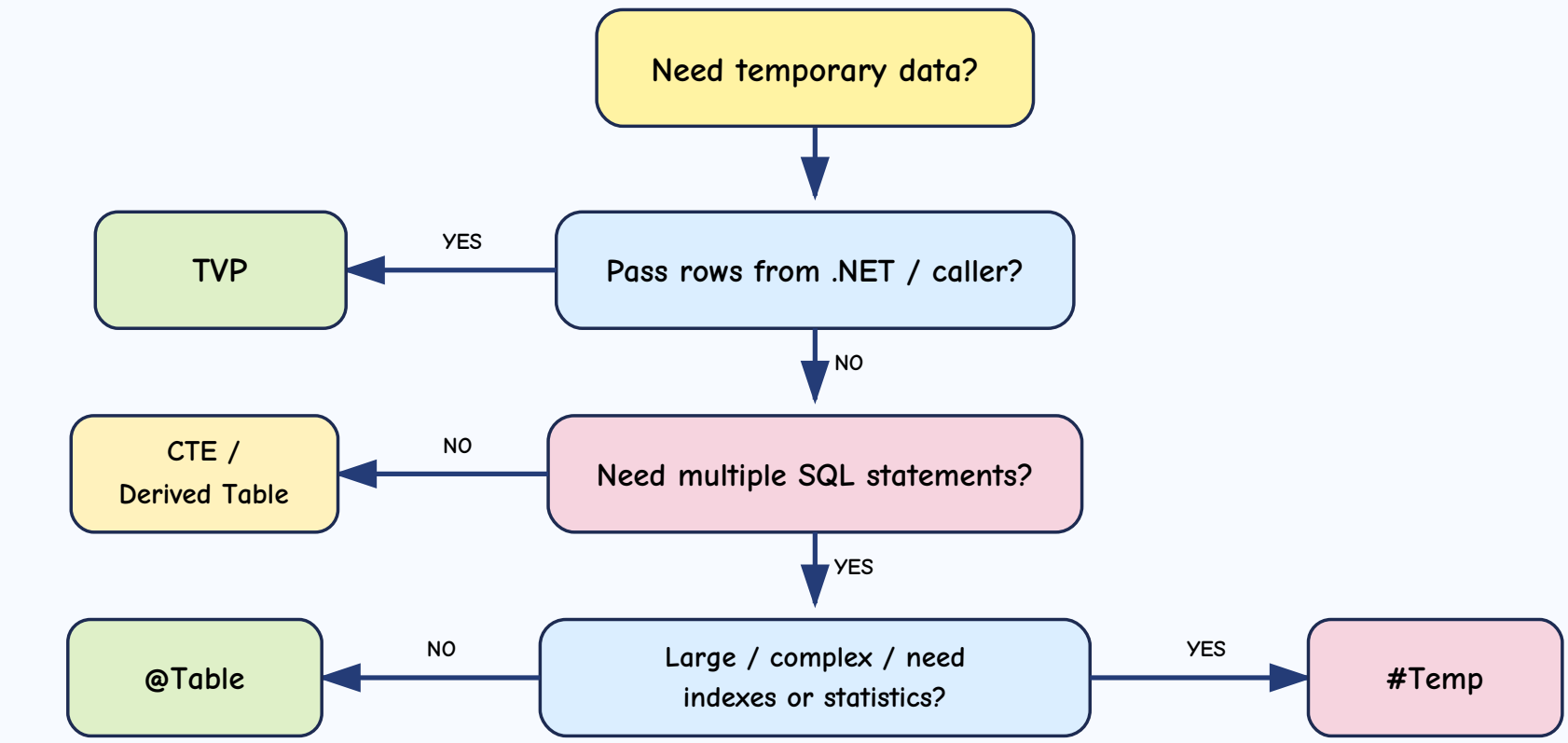
Need only a logical query expression → CTE

Need multiple statements → @TableVariable

Need one readable statement → CTE
```

Choosing the Right Temporary Structure

3D DIAGRAM Choosing the Right Temporary Structure



Special case: a few constant rows -> VALUES; cross-session sharing -> ##GlobalTemp.

Start with scope and reuse, then choose the structure based on row volume, indexing/statistics needs, and whether rows come from the application.

Use this practical decision flow:

EXAMPLE / FLOW

```
Need temporary data?
    →
Need to pass rows from application?
    → Yes → TVP

Need multiple SQL statements?
    → Yes → #TempTable or @TableVariable

Large/complex data or need statistics/indexes?
    → Yes → #TempTable

Small/simple working data?
    → Yes → @TableVariable

Only one SQL statement?
    → CTE or Derived Table

Need recursion?
```

```
    → Recursive CTE

Need a few hard-coded rows?
    → VALUES

Need to share between independent sessions?
    → Rare case → ##GlobalTempTable
```

What Is tempdb?

`tempdb` is a SQL Server system database used for temporary work.

It can contain:

EXAMPLE / FLOW

```
Local temporary tables
Global temporary tables
Temporary-table indexes
Table-variable storage
Work tables
Sort results
Hash-processing work
Spools
Other temporary engine operations
```

Microsoft describes `tempdb` as a global resource used for both user-created temporary objects and internal SQL Server processing.

---

tempdb Flow

EXAMPLE / FLOW

```
Query
    →
Needs Temporary Work
    →
tempdb
    →
Temporary Data / Sort / Hash / #Table
    →
Query Completes
```

Why tempdb Performance Matters

If many applications heavily use temporary objects, `tempdb` can become an important performance resource.

Examples include:

EXAMPLE / FLOW

```
Large #Temp tables
Large sorts
Large hash joins
Heavy GROUP BY
Many concurrent temporary tables
Version-store activity
```

Therefore, temporary structures should be useful-not created unnecessarily.

Statistics and Query Optimization

SQL Server's optimizer needs information about the data.

Conceptually:

EXAMPLE / FLOW

```
Statistics
    →
Estimated Rows
    →
Join Choice
    →
Memory Choice
    →
Index Choice
    →
Execution Plan
```

A poor estimate can produce a poor plan.

Temporary Table Statistics

Temporary tables are useful for complicated workloads because the optimizer gets stronger information about their data than it normally gets from table variables.

This is a major reason `#temp` tables often work better for larger intermediate results.

Table Variable Statistics

Table variables have no normal column statistics.

Deferred compilation improves the initial row-count estimate in modern compatibility levels, but SQL Server still does not receive full data-distribution statistics.

CTE Statistics

A CTE does not have independent statistics like a stored temporary table.

The optimizer works from the underlying tables and their statistics.

Example:

EXAMPLE / FLOW

```
CTE → fundraising.Gift → Gift statistics/indexes used by optimizer
```

Indexing Temporary Structures

Temporary Table

Best flexibility:

SQL

```sql
CREATE INDEX IX_GiftWork_ConstituentId
ON #GiftWork(ConstituentId);
```

Table Variable

More restricted:

SQL

```sql
DECLARE @GiftWork TABLE
(
    GiftId BIGINT PRIMARY KEY,
    ConstituentId BIGINT
);
```

CTE

You do not create an index directly on the CTE.

Instead, index the underlying permanent tables when appropriate.

Do Not Over-Index Temporary Tables

Suppose:

EXAMPLE / FLOW

```
Create #Temp
    →
Insert 1,000,000 rows
    →
Maintain 6 indexes
```

Every inserted row may create additional index-maintenance work.

Only create indexes that support important joins, filters, ordering, or grouping.

Scope and Session Behavior

Local Temp Table

EXAMPLE / FLOW

```
Connection A → #GiftWork → Connection A
Connection B → cannot normally access Connection A's #GiftWork
```

Global Temp Table

EXAMPLE / FLOW

```
Connection A → ##GiftWork ← Connection B
```

Table Variable

EXAMPLE / FLOW

```
Current batch/procedure → @GiftWork
```

CTE

EXAMPLE / FLOW

```
Current statement → CTE
```

Concurrency Considerations

Local temporary tables are usually safer for concurrent application requests because each session receives its own isolated temporary object.

Global temporary tables are shared and therefore introduce more concerns:

EXAMPLE / FLOW

```
Session A writes
        +
Session B reads/writes
        →
Possible coordination / naming / timing problems
```

For Web API request processing, local temporary tables are normally much safer than global temporary tables.

Temporary Tables in Concurrent Stored Procedures

Imagine 100 users call the same stored procedure.

Each session can create:

EXAMPLE / FLOW

```
#GiftWork
```

SQL Server internally separates those local temporary tables.

This is one reason local temp tables work well in application stored procedures.

Global Temporary Table Concurrency Problem

Suppose:

EXAMPLE / FLOW

```
User A → CREATE ##GiftWork
User B → CREATE ##GiftWork
```

Both are trying to use the same globally visible name.

This can produce conflicts or unexpected shared data.

Therefore, ## tables should be rare in normal Web API business processing.

Performance Trade-Off - Temporary Table

Advantages

EXAMPLE / FLOW

```
Statistics
Indexes
Large intermediate results
Multiple statements
Complex joins
Flexible schema
```

Costs

EXAMPLE / FLOW

```
tempdb usage
Create/drop work
Index-maintenance work
Possible recompilation
```

Best for medium/large or complex intermediate processing.

Performance Trade-Off - Table Variable

Advantages

EXAMPLE / FLOW

```
Simple declaration
Short scope
Convenient for small working sets
Useful inside procedures/functions where supported
```

Limitations

EXAMPLE / FLOW

```
No normal column statistics
Less index flexibility
Can produce weaker plans for complicated data
```

Best for relatively small and straightforward work.

Performance Trade-Off - CTE

Advantages

EXAMPLE / FLOW

```
Readable
No explicit temp-table management
Excellent for one statement
Recursive capability
```

Limitations

EXAMPLE / FLOW

```
Not reusable across statements
No direct indexes
No independent stored statistics
Repeated references can cause repeated work
```

Best for logical query organization.

Performance Trade-Off - Derived Table

Advantages

EXAMPLE / FLOW

```
Simple
Compact
Good for one inline calculation
```

Limitations

EXAMPLE / FLOW

```
Only query scope
Can become hard to read when deeply nested
No reusable physical object
```

Best for short inline logic.

Performance Trade-Off - TVP

Advantages

EXAMPLE / FLOW

```
Send many rows in one call
Strongly typed
Useful with .NET
Reduces many individual parameters/round trips
```

Limitations

EXAMPLE / FLOW

```
READONLY
No normal column statistics
Less flexible for complicated intermediate processing
```

If complex processing is required:

EXAMPLE / FLOW

```
.NET → TVP → #TempTable → Processing
```

Performance Trade-Off - VALUES

Advantages

EXAMPLE / FLOW

```
Very simple
Excellent for a few constant rows
No separate object creation
```

Limitations

EXAMPLE / FLOW

```
Not suitable for large changing business datasets
Hard to maintain if many values are hard-coded
```

Use it for small static row sets.

Practical FundraisingDb Example

Requirement:

> ! Produce a report of constituents who donated more than $5,000 and show their gift count.

A useful temporary-table solution:

SQL

```sql
DROP TABLE IF EXISTS #ConstituentGiftSummary;

CREATE TABLE #ConstituentGiftSummary
(
    ConstituentId BIGINT NOT NULL PRIMARY KEY,
    GiftCount INT NOT NULL,
    TotalGiving DECIMAL(19,4) NOT NULL
);

INSERT INTO #ConstituentGiftSummary
(
    ConstituentId,
    GiftCount,
    TotalGiving
)
SELECT
    ConstituentId,
    COUNT(*) AS GiftCount,
    SUM(TotalAmount) AS TotalGiving
FROM fundraising.Gift
GROUP BY ConstituentId
HAVING SUM(TotalAmount) > 5000;

SELECT
    c.ConstituentId,
    c.DisplayName,
    s.GiftCount,
    s.TotalGiving
FROM #ConstituentGiftSummary AS s
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = s.ConstituentId
ORDER BY s.TotalGiving DESC;

DROP TABLE #ConstituentGiftSummary;
```

Same Requirement Using a CTE

SQL

```sql
;WITH ConstituentGiftSummary AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS TotalGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
    HAVING SUM(TotalAmount) > 5000
)
SELECT
    c.ConstituentId,
    c.DisplayName,
    s.GiftCount,
```

```
    s.TotalGiving
FROM ConstituentGiftSummary AS s
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = s.ConstituentId
ORDER BY s.TotalGiving DESC;
```

For this one-use report, the CTE may be simpler.

If the intermediate results must be reused and indexed several times, `#ConstituentGiftSummary` becomes more useful.

Common Beginner Mistakes

Mistake 1

Thinking table variables are always stored only in RAM.

Correct idea:

EXAMPLE / FLOW

```
@TableVariable can use tempdb resources.
```

Mistake 2

Using a global temp table when local scope is enough.

Prefer:

EXAMPLE / FLOW

```
#GiftWork
```

instead of:

EXAMPLE / FLOW

```
##GiftWork
```

for normal request processing.

Mistake 3

Using a CTE for data needed in several independent SQL statements.

Use a temporary table instead.

Mistake 4

Creating many indexes before checking whether they are useful.

Indexes have both benefits and maintenance cost.

Mistake 5

Automatically assuming `@TableVariable` is faster than `#TempTable`.

Performance depends on row count, statistics, indexes, query shape, and execution plan.

Best Practices

Use `#temp` for larger or complicated intermediate workloads.

Use `@table` for simple, relatively small temporary data where its limitations are acceptable.

Use CTEs for readable single-statement logic and recursion.

Use derived tables for small inline query logic.

Use TVPs when a .NET/API caller must send a set of rows to SQL Server.

Use `VALUES` for small constant row sets.

Avoid global temporary tables unless sharing across sessions is an actual requirement.

Measure performance using the real execution plan rather than relying only on general rules.

Final Memory Diagram

EXAMPLE / FLOW

```
Need Temporary Data
        →
One Statement?
        → CTE / Derived Table

Multiple Statements?
        →
Small + Simple?
        → @TableVariable

Large / Complex / Need Index + Statistics?
        → #TemporaryTable

Pass Rows from .NET?
        → TVP

Few Constant Rows?
        → VALUES

Share Across Sessions?
        → ##GlobalTemporaryTable
```

Final Memory Rule

EXAMPLE / FLOW

```
#Temp     = powerful temporary table

##Temp    = shared temporary table

@Table    = temporary table-shaped variable

CTE       = named query for one statement

Derived   = query inside FROM

TVP       = send multiple rows as a parameter

VALUES    = small inline row set

tempdb    = SQL Server temporary workspace
```